HDOC- 34

Aim:

To write a menu-driven C program using separate functions to verify:

1. Left Distributive Property

   $$A(B+C)=(AB)+(AC)$$

2. Right Distributive Property

   $$(A+B)C=(AC)+(BC)$$

Task 1: Algorithm:

Step 1: Start the program.

Step 2: Display the menu:

- Left distributive property
- Right distributive property
- Exit

Step 3: Read the user's choice.

Step 4 — Left distributive property: Read m, n, and p and declare:

$$A_{m\times n},\quad B_{n\times p},\quad C_{n\times p}$$

Calculate:

$$LHS=A(B+C)$$

and

$$RHS=(AB)+(AC)$$

Display both matrices and compare them element by element. If they are equal, the left distributive property is verified.

Step 5 — Right distributive property: Read m, n, and p and declare:

$$A_{m\times n},\quad B_{m\times n},\quad C_{n\times p}$$

Calculate:

\[LHS=(A+B)C \]

and

\[RHS=(AC)+(BC) \]

Display both matrices and compare them element by element. If they are equal, the right distributive property is verified.

Step 6: Repeat the menu until the user selects 3 (Exit).

Step 7: Stop.

Task 2: Test case table:

Test Case	Choice	Dimensions (m, n, p)	Matrix A	Matrix B	Matrix C	Expected Result
1	1 – Left Distributive	2, 2, 2	1 2 3 4	5 6 7 8	1 2 3 4	A(B+C) = (AB)+(AC) ⟶ Left distributive property verified
2	1 – Left Distributive	2, 3, 2	1 2 3 4 5 6	1 2 3 4 5 6	6 5 4 3 2 1	Both sides produce the same 2 × 2 matrix ⟶ Left distributive property verified
3	2 – Right Distributive	2, 2, 2	1 2 3 4	5 6 7 8	1 0 0 1	(A+B)C = AC+BC ⟶ Right distributive property verified
4	2 – Right Distributive	2, 3, 2	1 2 3 4 5 6	6 5 4 3 2 1	1 2 3 4 5 6	Both sides produce the same 2 × 2 matrix ⟶ Right distributive property verified
5	3 – Exit	—	—	—	—	Program terminated

TASK 3: C PROGRAM:

```
#include <stdio.h>
/* Function to read a matrix */
void readMatrix(int r, int c, int M[r][c])
{
   int i, j;
   for (i = 0; i < r; i++)
   {
      for (j = 0; j < c; j++)
      {
         scanf("%d", &M[i][j]);
      }
   }
}
/* Function to display a matrix */
void displayMatrix(int r, int c, int M[r][c])
{
   int i, j;
   for (i = 0; i < r; i++)
   {
      for (j = 0; j < c; j++)
      {
         printf("%d\t", M[i][j]);
      }
      printf("\n");
   }
}
/* Function to add two matrices */
void addMatrix(int r, int c,
               int A[r][c],
               int B[r][c],
               int result[r][c])
{
   int i, j;
   for (i = 0; i < r; i++)
   {
      for (j = 0; j < c; j++)
      {
         result[i][j] = A[i][j] + B[i][j];
      }
   }
}
/* Function to multiply two matrices */
```

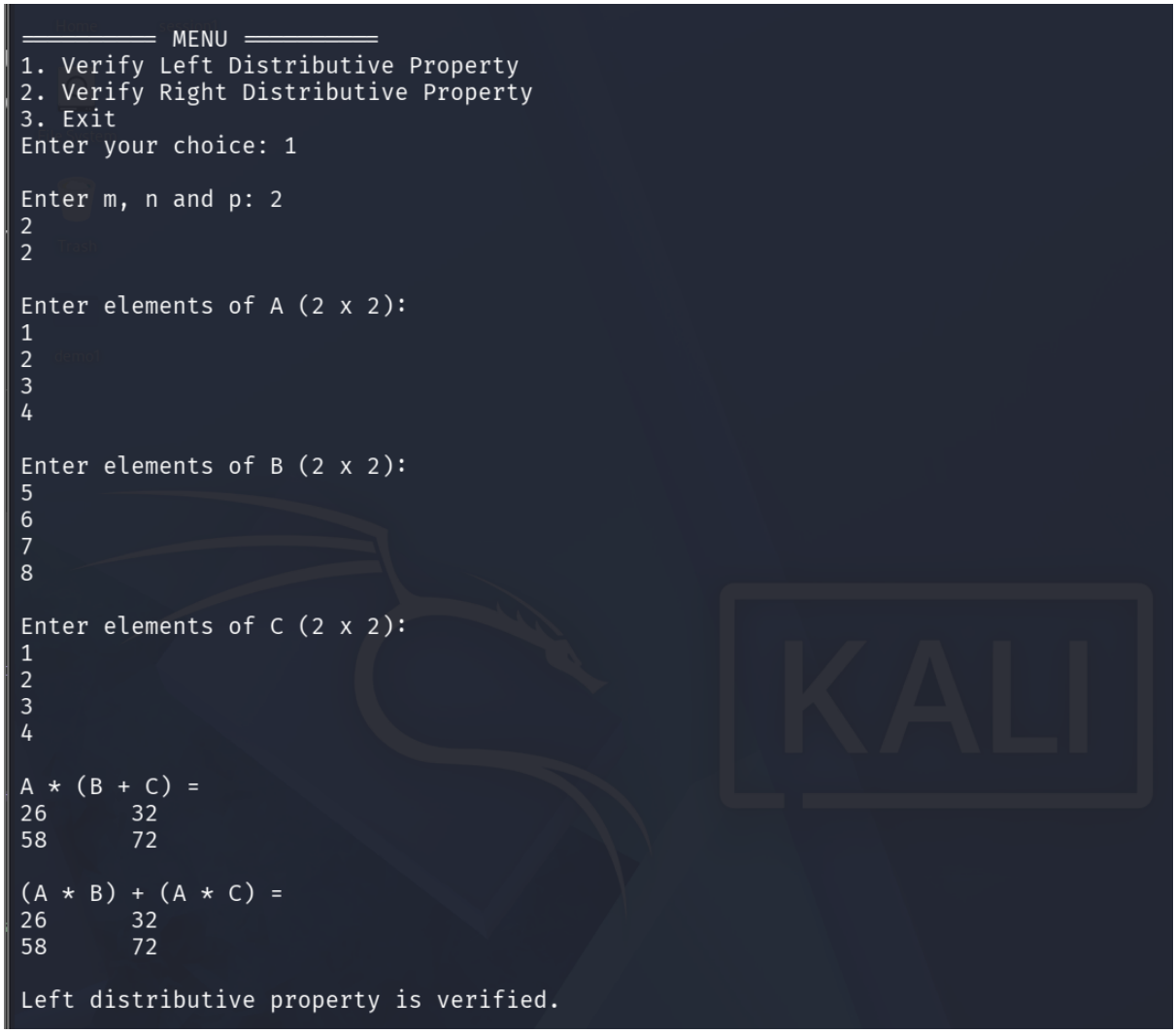
```
void multiplyMatrix(int r1, int c1, int c2,
                    int A[r1][c1],
                    int B[c1][c2],
                    int result[r1][c2])
{
   int i, j, k;
   for (i = 0; i < r1; i++)
   {
      for (j = 0; j < c2; j++)
      {
         result[i][j] = 0;
         for (k = 0; k < c1; k++)
         {
            result[i][j] += A[i][k] * B[k][j];
         }
      }
   }
}
/* Function to compare two matrices */
int compareMatrix(int r, int c,
                  int A[r][c],
                  int B[r][c])
{
   int i, j;
   for (i = 0; i < r; i++)
   {
      for (j = 0; j < c; j++)
      {
         if (A[i][j] != B[i][j])
         {
            return 0;
         }
      }
   }
   return 1;
}
/* Function to verify left distributive property */
void leftDistributive()
{
   int m, n, p;
   printf("\nEnter m, n and p: ");
   scanf("%d %d %d", &m, &n, &p);
   if (m <= 0 || n <= 0 || p <= 0)
   {
      printf("Invalid dimensions.\n");
      return;
```

```
    }
    int A[m][n];
    int B[n][p];
    int C[n][p];
    int BplusC[n][p];
    int LHS[m][p];
    int AB[m][p];
    int AC[m][p];
    int RHS[m][p];
    printf("\nEnter elements of A (%d x %d):\n", m, n);
    readMatrix(m, n, A);
    printf("\nEnter elements of B (%d x %d):\n", n, p);
    readMatrix(n, p, B);
    printf("\nEnter elements of C (%d x %d):\n", n, p);
    readMatrix(n, p, C);
    /* Calculate A(B + C) */
    addMatrix(n, p, B, C, BplusC);
    multiplyMatrix(m, n, p, A, BplusC, LHS);
    /* Calculate AB + AC */
    multiplyMatrix(m, n, p, A, B, AB);
    multiplyMatrix(m, n, p, A, C, AC);
    addMatrix(m, p, AB, AC, RHS);
    printf("\nA * (B + C) =\n");
    displayMatrix(m, p, LHS);
    printf("\n(A * B) + (A * C) =\n");
    displayMatrix(m, p, RHS);
    if (compareMatrix(m, p, LHS, RHS))
    {
        printf("\nLeft distributive property is verified.\n");
    }
    else
    {
        printf("\nLeft distributive property is NOT verified.\n");
    }
}
/* Function to verify right distributive property */
void rightDistributive()
{
    int m, n, p;
    printf("\nEnter m, n and p: ");
    scanf("%d %d %d", &m, &n, &p);
    if (m <= 0 || n <= 0 || p <= 0)
    {
        printf("Invalid dimensions.\n");
        return;
    }
```

```
    int A[m][n];
    int B[m][n];
    int C[n][p];
    int AplusB[m][n];
    int LHS[m][p];
    int AC[m][p];
    int BC[m][p];
    int RHS[m][p];
    printf("\nEnter elements of A (%d x %d):\n", m, n);
    readMatrix(m, n, A);
    printf("\nEnter elements of B (%d x %d):\n", m, n);
    readMatrix(m, n, B);
    printf("\nEnter elements of C (%d x %d):\n", n, p);
    readMatrix(n, p, C);
    /* Calculate (A + B)C */
    addMatrix(m, n, A, B, AplusB);
    multiplyMatrix(m, n, p, AplusB, C, LHS);
    /* Calculate AC + BC */
    multiplyMatrix(m, n, p, A, C, AC);
    multiplyMatrix(m, n, p, B, C, BC);
    addMatrix(m, p, AC, BC, RHS);
    printf("\n(A + B) * C =\n");
    displayMatrix(m, p, LHS);
    printf("\n(A * C) + (B * C) =\n");
    displayMatrix(m, p, RHS);
    if (compareMatrix(m, p, LHS, RHS))
    {
        printf("\nRight distributive property is verified.\n");
    }
    else
    {
        printf("\nRight distributive property is NOT verified.\n");
    }
}
int main()
{
    int choice;
    do
    {
        printf("\n========== MENU ==========\n");
        printf("1. Verify Left Distributive Property\n");
        printf("2. Verify Right Distributive Property\n");
        printf("3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice)
```

```
    {
        case 1:
            leftDistributive();
            break;
        case 2:
            rightDistributive();
            break;
        case 3:
            printf("Program terminated.\n");
            break;
        default:
            printf("Invalid choice. Please enter 1, 2 or 3.\n");
    }
  } while (choice != 3);
  return 0;
}
```

Task 4: Output:



```
═══════ MENU ═══════
1. Verify Left Distributive Property
2. Verify Right Distributive Property
3. Exit
Enter your choice: 1

Enter m, n and p: 2
3
2

Enter elements of A (2 x 3):
1
2
3
4
5
6

Enter elements of B (3 x 2):
1
2
3
4
5
6

Enter elements of C (3 x 2):
6
5
4
3
2
1

A * (B + C) =
42      42
105     105

(A * B) + (A * C) =
42      42
105     105

Left distributive property is verified.
```

```
═══════ MENU ═══════
1. Verify Left Distributive Property
2. Verify Right Distributive Property
3. Exit
Enter your choice: 2

Enter m, n and p: 2
2
2

Enter elements of A (2 x 2):
1
2
3
4

Enter elements of B (2 x 2):
5
6
7
8

Enter elements of C (2 x 2):
1
0
0
1

(A + B) * C =
6       8
10      12

(A * C) + (B * C) =
6       8
10      12

Right distributive property is verified.
```

```
========== MENU ==========
1. Verify Left Distributive Property
2. Verify Right Distributive Property
3. Exit
Enter your choice: 2

Enter m, n and p: 2
3
2

Enter elements of A (2 x 3):
1
2
3
4
5
6

Enter elements of B (2 x 3):
6
5
4
3
2
1

Enter elements of C (3 x 2):
1
2
3
4
5
6

(A + B) * C =
63      84
63      84

(A * C) + (B * C) =
63      84
63      84

Right distributive property is verified.

========== MENU ==========
1. Verify Left Distributive Property
2. Verify Right Distributive Property
3. Exit
Enter your choice: 3
Program terminated.
```